

TECHNICAL DOCUMENTATION

Superstore Data Warehouse

Batch ELT pipeline with Apache Airflow, dbt and DuckDB

Author: Mina Milad Matta

Program: DataOps Engineering, NTI

Version: 1.0, October 2026

Repository: https://github.com/MinaMiladd/MinaMiladd-Superstore_Data_Warehouse_Airflow_dbt_DuckDB.git

Stack: Apache Airflow 3.1 · dbt-core 1.12 · DuckDB 1.5 · Python 3.11 · Docker

Contents

1. Overview	1
2. Source Data	1
3. Data Lineage	1
4. Layer Details	2
4.1 ODS — ods.superstore	2
4.2 Staging — main_stg.stg_superstore	2
4.3 Marts — main_dwh.*	2
5. Data Dictionary	3
fact_orders	3
dim_customer	3
dim_product	3
dim_location	4
dim_date	4
6. Orchestration	4
7. Data Quality & Testing	5
8. Infrastructure	6
Container	6
Volume mounts	6
Why a separate dbt virtualenv?	7
9. Configuration Reference	7
my_project/dbt_project.yml	7
my_project/profiles.yml	7
Airflow environment (docker-compose.yml)	7
10. Operations Runbook	8
11. Troubleshooting	8
12. Known Limitations	9

This document covers the internals of the project: data lineage, the data dictionary, transformation logic, orchestration, testing, operations and known limitations. For a quick overview and setup, see the README on GitHub.

1. Overview

Item	Value
Pattern	Batch ELT, full refresh
Orchestrator	Apache Airflow 3.1 (standalone mode, Python 3.11)
Transformation	dbt-core 1.12.5 with dbt-duckdb 1.11.0
Storage / compute	DuckDB 1.5.6, a single embedded file (<code>my_project/dev.duckdb</code>)
Modeling approach	Kimball dimensional model (star schema)
Schedule	Daily (@daily, UTC)
Runtime	One Docker container

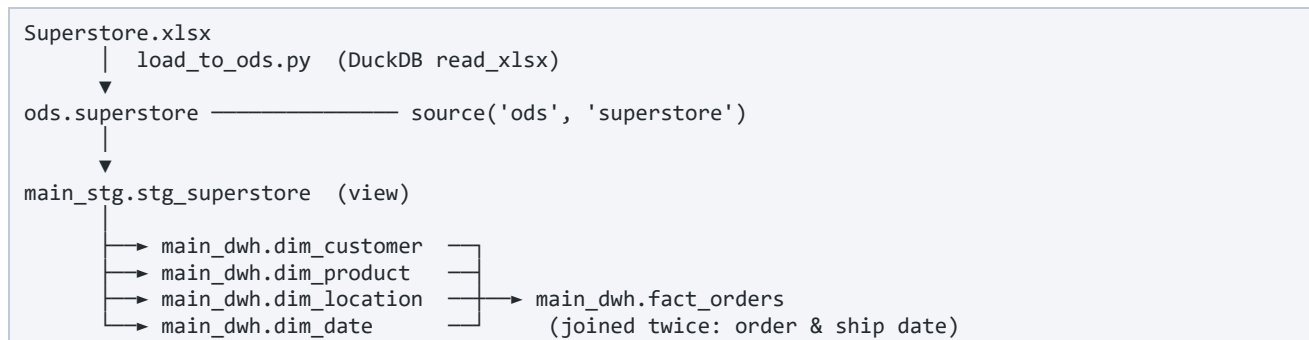
Business questions it answers: sales and profit by product category, customer segment, region and time period; shipping lead time (order date to ship date); discount impact on profitability.

2. Source Data

Property	Value
File	<code>data/raw/Superstore.xlsx</code>
Sheet	<code>Sample - Superstore</code>
Rows	9,994 order line items
Orders	5,009 distinct <code>Order ID</code> s
Date range	Orders 2014-01-03 → 2017-12-30, shipments through 2018-01-05
Totals	≈ 2.30 M sales, ≈ 286 K profit
Grain	One row per product per order

Known source quirk: a single `Product ID` can appear with more than one `Product Name`. The data model works around this (see `dim_product`).

3. Data Lineage



The same lineage, as rendered by dbt docs:



Figure 1: dbt lineage graph. The `ods.superstore` source (green) feeds the staging view, which feeds four dimensions and the fact table.

To explore it yourself, run `dbt docs generate` and then `dbt docs serve --port 8081` from `my_project/`. Port 8081 avoids a clash with Airflow on 8080.

4. Layer Details

4.1 ODS — `ods.superstore`

- **Built by:** `scripts/load_to_ods.py`
- **Method:** `CREATE OR REPLACE TABLE ... AS SELECT * FROM read_xlsx(...)` using DuckDB's `excel` extension
- **Behavior:** full reload on every run, so it is idempotent and safe to re-run
- **Columns:** kept exactly as in the spreadsheet, including spaces in names (e.g. `"Order ID"`)

4.2 Staging — `main_stg.stg_superstore`

- **Materialization:** view
- **Transformations:**
 - Renames every column to `snake_case`
 - Casts `Order Date` / `Ship Date` to `DATE`
 - No filtering or aggregation, so the grain matches the source (9,994 rows)
- **Purpose:** the single cleaned interface that every mart model reads from. No mart model reads the ODS directly.

4.3 Marts — `main_dwh.*`

- **Materialization:** table (rebuilt on every `dbt run`)
- **Surrogate keys:** `md5()` hashes of the natural or business key, so keys are deterministic across rebuilds

5. Data Dictionary

fact_orders

Grain: **one row per order line item** (9,994 rows).

Column	Type	Description
<code>sales_key</code>	double	Primary key (source <code>Row ID</code>)
<code>order_id</code>	varchar	Degenerate dimension: the order number
<code>order_date_key</code>	integer	FK → <code>dim_date.date_key</code> (order date)
<code>ship_date_key</code>	integer	FK → <code>dim_date.date_key</code> (ship date)
<code>customer_key</code>	varchar	FK → <code>dim_customer.customer_key</code>
<code>product_key</code>	varchar	FK → <code>dim_product.product_key</code>
<code>geography_key</code>	varchar	FK → <code>dim_location.geography_key</code>
<code>ship_mode</code>	varchar	First Class / Second Class / Standard Class / Same Day
<code>sales</code>	double	Line revenue
<code>quantity</code>	double	Units ordered
<code>discount</code>	double	Discount rate (0–1)
<code>profit</code>	double	Line profit (can be negative)

dim_customer

793 rows, one per customer.

Column	Type	Description
<code>customer_key</code>	varchar	PK, <code>md5(customer_id)</code>
<code>customer_id</code>	varchar	Business key
<code>customer_name</code>	varchar	Customer full name
<code>segment</code>	varchar	Consumer / Corporate / Home Office

dim_product

1,894 rows, one per distinct (`product_id`, `product_name`) pair.

Column	Type	Description
<code>product_key</code>	varchar	PK, <code>md5(product_id product_name)</code>
<code>product_id</code>	varchar	Business key (not unique on its own)
<code>product_name</code>	varchar	Product description
<code>category</code>	varchar	Furniture / Office Supplies / Technology
<code>sub_category</code>	varchar	e.g. Chairs, Phones, Binders

Why a composite key? Some `product_ids` carry several names in the source. Keying on the ID alone would merge different products, or create duplicate dimension rows that fan out the fact join.

dim_location

632 rows, one per distinct location.

Column	Type	Description
geography_key	varchar	PK, md5(country region state city postal_code)
country	varchar	Country
region	varchar	Central / East / South / West
state	varchar	State
city	varchar	City
postal_code	double	Postal code

dim_date

1,464 rows: every calendar day from the first order date to the last ship date.

Column	Type	Description
date_key	integer	PK, YYYYMMDD (e.g. 20170315)
date_day	date	Calendar date
year	bigint	Year
quarter	bigint	Quarter (1–4)
month	bigint	Month number (1–12)
month_name	varchar	e.g. March
day_of_month	bigint	Day (1–31)
day_name	varchar	e.g. Wednesday
is_weekend	boolean	true on Saturday and Sunday

dim_date is a **role-playing dimension**: fact_orders joins to it twice, once as order date and once as ship date.

6. Orchestration

DAG: [airflow/dags/superstore_dwh_dag.py](#)

Setting	Value
dag_id	superstore_dwh
schedule	@daily
start_date	2026-10-01 UTC
catchup	False
retries	1, with a 2-minute delay
Paused on create	Yes (DAGS_ARE_PAUSED_AT_CREATION)

Task flow (strictly sequential):

```
dbt_debug → load_to_ods → dbt_test_sources → dbt_run → dbt_test
```

#	Task	Command (simplified)	Fails when
1	<code>dbt_debug</code>	<code>dbt debug</code>	Profile, project or DB is unreachable
2	<code>load_to_ods</code>	<code>python scripts/load_to_ods.py</code>	Excel file is missing or DB is locked
3	<code>dbt_test_sources</code>	<code>dbt test --select source:ods --indirect-selection cautious</code>	Raw source tests fail
4	<code>dbt_run</code>	<code>dbt run</code>	A model has a SQL error
5	<code>dbt_test</code>	<code>dbt test</code>	Any data quality test fails

Every dbt command runs from `/opt/airflow/project/my_project` with `--profiles-dir . --target dev`, so the project uses its own committed `profiles.yml`.

`--indirect-selection cautious` on task 3 keeps the source check from also running tests on models that haven't been built yet.

`--no-partial-parse` makes dbt re-parse the project on every run. `my_project/target/` is shared with the host through the volume mount, and a parse cache written by dbt on Windows contains Windows paths that crash dbt on Linux.

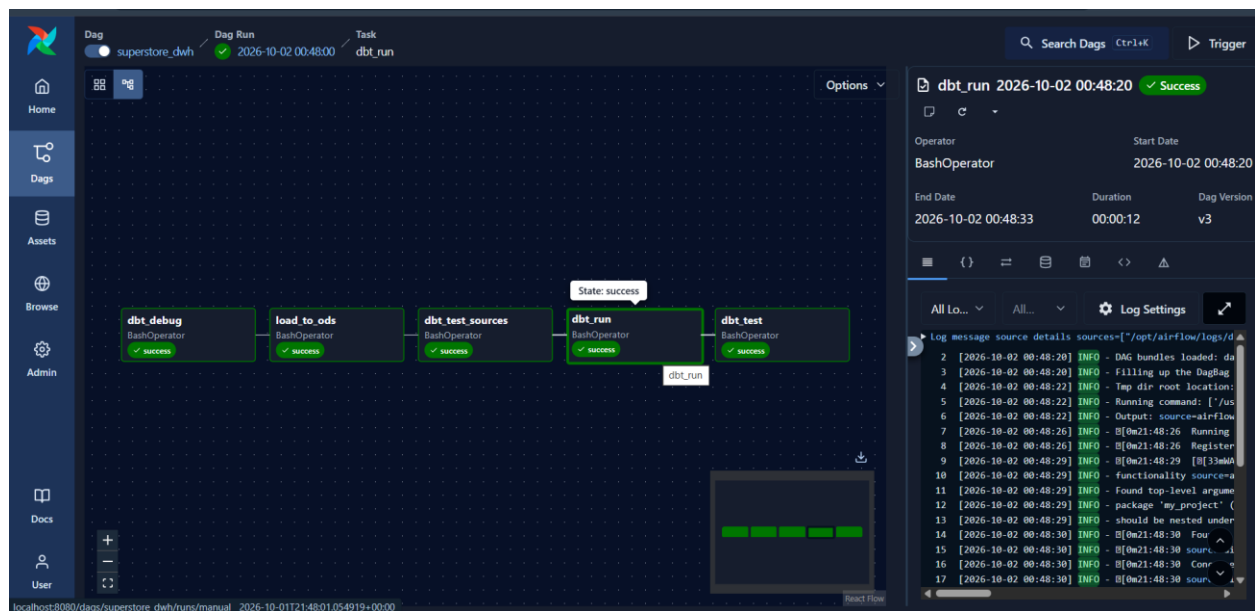


Figure 2: a successful `superstore_dwh` run in the Airflow 3 Graph view, with all five tasks green.

7. Data Quality & Testing

20 tests in total, run by the `dbt_test` task.

Model	Column	Tests
<code>stg_superstore</code>	<code>row_id</code>	<code>unique, not_null</code>
<code>stg_superstore</code>	<code>order_id, order_date, customer_id, product_id, sales, quantity</code>	<code>not_null</code>
<code>fact_orders</code>	<code>sales_key</code>	<code>unique, not_null</code>
<code>fact_orders</code>	<code>customer_key, product_key, geography_key, order_date_key, ship_date_key</code>	<code>not_null, relationships</code>

Together, the `relationships` and `not_null` tests on the fact table guarantee that **every fact row joins to exactly one member of each dimension**. This catches fan-out or orphaned rows caused by key logic changes.

Tests appear in dbt docs next to each column (**U** = unique, **N** = not_null, **F** = relationships / foreign key):

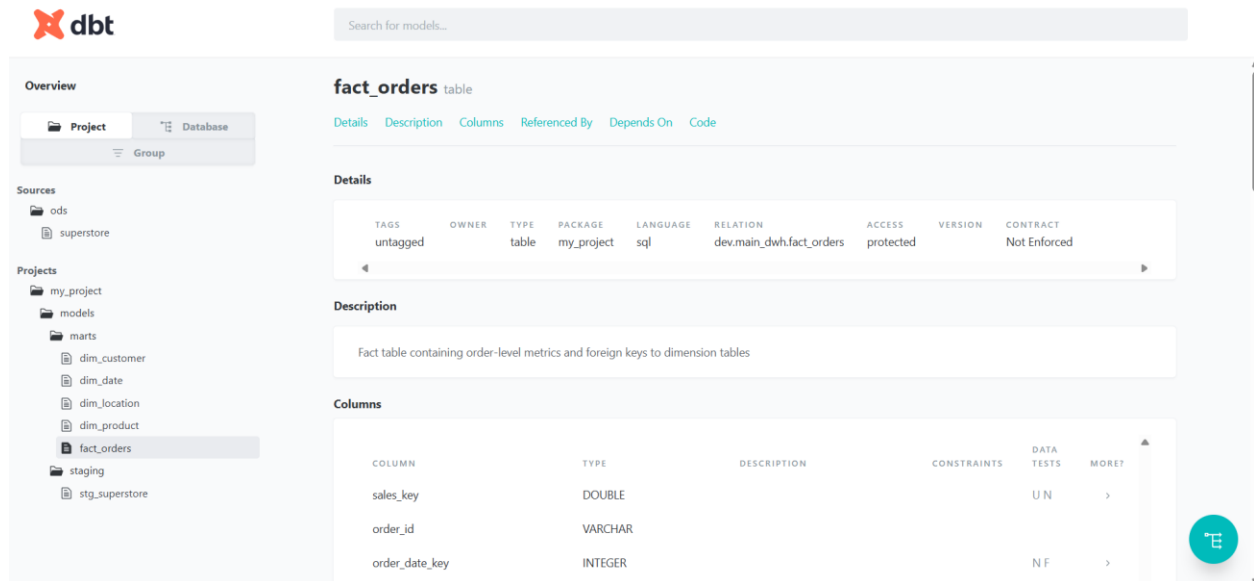


Figure 3: the dbt docs page for `fact_orders`, showing that it is built in `dev.main_dwh` and listing the tests on each column.

Run the tests manually:

```
cd my_project
dbt test --profiles-dir . --target dev          # all tests
dbt test --profiles-dir . --target dev --select fact_orders # one model
```

8. Infrastructure

Container

Component	Detail
Base image	<code>apache/airflow:3.1.0-python3.11</code>
Airflow mode	<code>standalone</code> (API server, scheduler, DAG processor and triggerer in one process)
dbt runtime	Separate virtualenv at <code>/opt/dbt_venv</code>
DuckDB extension	<code>excel</code> , pre-installed at build time
Port	<code>8080</code> → Airflow UI

Volume mounts

Host path	Container path	Purpose
<code>airflow/dags</code>	<code>/opt/airflow/dags</code>	DAG files (picked up live)
<code>airflow/logs</code>	<code>/opt/airflow/logs</code>	Task logs, readable on the host
<code>.</code> (repo root)	<code>/opt/airflow/project</code>	dbt project, scripts and data

Because the repo root is mounted, changes to dbt models, scripts or the DAG take effect on the next run **without rebuilding the image**. Only changes to the `Dockerfile` or `requirements-dbt.txt` need `docker compose up -d --build`.

Why a separate dbt virtualenv?

Airflow pins a large dependency tree. Installing dbt into the same environment risks version conflicts (e.g. `protobuf`, `jinja2`). A dedicated venv, called by its absolute path (`/opt/dbt_venv/bin/dbt`), keeps the two isolated so each can be upgraded independently.

9. Configuration Reference

my_project/dbt_project.yml

```
models:
  my_project:
    staging:
      +schema: stg
      +materialized: view
    marts:
      +schema: dwh
      +materialized: table
```

dbt's default `generate_schema_name` joins the target schema (`main`) and the custom schema, which gives `main_stg` and `main_dwh`.

my_project/profiles.yml

```
my_project:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: dev.duckdb # relative to my_project/
      threads: 1
```

`threads: 1` is deliberate, because DuckDB allows a single writer.

Airflow environment (docker-compose.yml)

Variable	Value	Effect
<code>AIRFLOW__CORE__LOAD_EXAMPLES</code>	<code>False</code>	Hides the example DAGs
<code>AIRFLOW__CORE__SIMPLE_AUTH_MANAGER_ALL_ADMINS</code>	<code>True</code>	No login (local dev only)
<code>AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION</code>	<code>True</code>	New DAGs start paused
<code>DO_NOT_TRACK</code>	<code>1</code>	Turns off dbt anonymous usage stats

10. Operations Runbook

Task	Command / action
Start the stack	<code>cd airflow && docker compose up -d --build</code>
Stop the stack	<code>cd airflow && docker compose down</code>
Trigger a run	Airflow UI → <code>superstore_dwh</code> → Trigger
Re-run from a failed task	Airflow UI → failed task → Clear (downstream included)
Rebuild the warehouse from scratch	Delete <code>my_project/dev.duckdb</code> , then trigger the DAG
Run dbt in the container	<code>docker exec -it superstore-airflow bash -c "cd /opt/airflow/project/my_project && /opt/dbt_venv/bin/dbt build --profiles-dir . --target dev"</code>
View task logs on the host	<code>airflow/logs/dag_id=superstore_dwh/...</code>
Update the source data	Replace <code>data/raw/Superstore.xlsx</code> (same sheet name), then trigger the DAG

11. Troubleshooting

Symptom (in task log)	Cause	Fix
<code>cd: /opt/airflow/project/...: No such file or directory</code>	<code>DBT_DIR</code> in the DAG doesn't match the dbt folder name	Set <code>DBT_DIR</code> to <code>{PROJECT_DIR}/my_project</code>
<code>Could not find profile named 'my_project'</code>	Profile name in <code>profiles.yml</code> doesn't match <code>dbt_project.yml</code> (case-sensitive)	Make both exactly <code>my_project</code>
<code>mapping values are not allowed in this context</code>	YAML indentation error. dbt strips leading whitespace from the file, so a fully indented file breaks	Start top-level keys (<code>version</code> , <code>models</code>) at column 0
<code>Configuration paths exist ... which do not apply to any resources</code>	The key under <code>models:</code> isn't the project name	Use <code>models: my_project:</code>
<code>IO Error: Could not set lock on file "dev.duckdb"</code>	Another process (DBBeaver, CLI, notebook) has the DB open	Close that connection, then clear the task
<code>Nothing to do in dbt_test_sources</code>	No tests are defined on the source	Expected until source tests are added (see Limitations)
<code>'dbt_duckdb://macros/columns.sql'</code> <code>KeyError</code> during parsing	<code>target/partial_parse.msgpack</code> was written by dbt on Windows (backslash paths) and reused by dbt in the container	Delete the file. The DAG passes <code>--no-partial-parse</code> to prevent this
<code>dbt docs serve</code> fails, or opens Airflow instead	Both default to port 8080	<code>dbt docs serve --port 8081</code>
DAG doesn't appear in the UI	Python error in the DAG file	Check <code>airflow/logs/dag_processor/...</code> , or the import errors shown in the UI

12. Known Limitations

Limitation	Impact	Possible improvement
Full refresh on every run	Fine at 10 K rows, but doesn't scale	Incremental <code>fact_orders</code> (<code>materialized='incremental'</code>)
No tests on the raw source	<code>dbt_test_sources</code> currently does nothing	Add <code>not_null</code> / <code>unique</code> on "Row ID" in <code>sources.yml</code>
Row ID / Postal Code typed as <code>double</code>	Inherited from Excel inference	Cast to <code>integer</code> / <code>varchar</code> in staging
Customer attributes use <code>max()</code>	No history of changes to segment or name	SCD Type 2 with dbt snapshots
Schemas named <code>main_stg</code> / <code>main_dwh</code>	Prefix comes from dbt's default naming	Custom <code>generate_schema_name</code> macro
Local, single-container setup	No auth, no high availability	CeleryExecutor or Kubernetes, with a managed warehouse
Deprecated generic test syntax	dbt shows warnings (5 occurrences)	Move test arguments under <code>arguments:</code>